

HACKYEAH 2026 • AI SAFETY INFRASTRUCTURE

# AI CONTROL LAYER

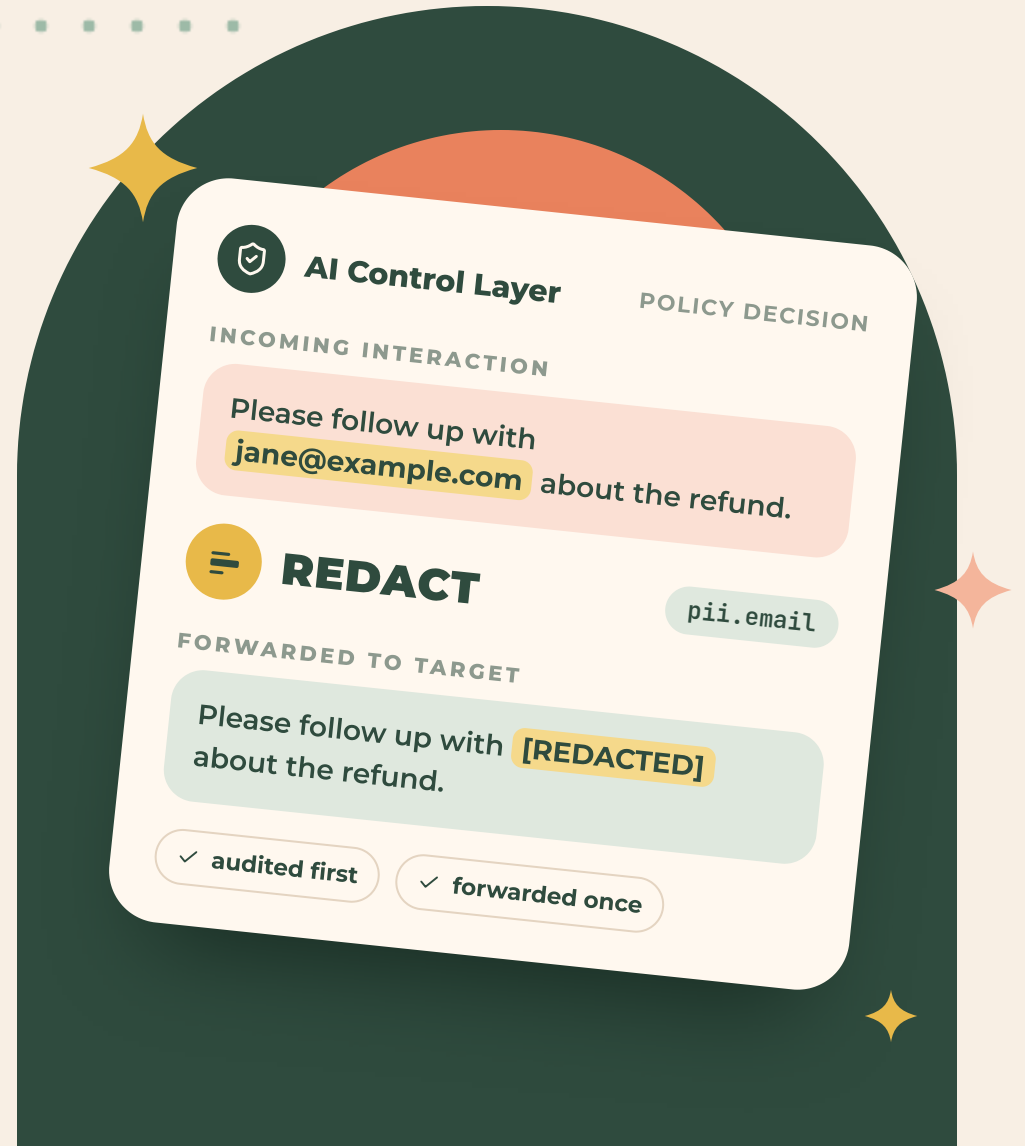
*built locally, enforced centrally*

A provider-independent control layer that checks

**every AI interaction**

before it reaches a model, agent, tool or API.

✦ Every decision is explained clearly.



# What can go wrong:

## Sensitive data

PII, credentials and private-key material can move into model or tool paths.

## Unsafe intent

Prompt injection, instruction override and exfiltration intent need policy decisions.

## Blind operations

Without safe evidence, operators cannot explain what happened or scale trust.

## AI traffic is the new **production boundary**.

- ✓ **every prompt**
- ✓ **every tool call**
- ✓ **every model response**

...can carry **sensitive data** or **risky instructions**.

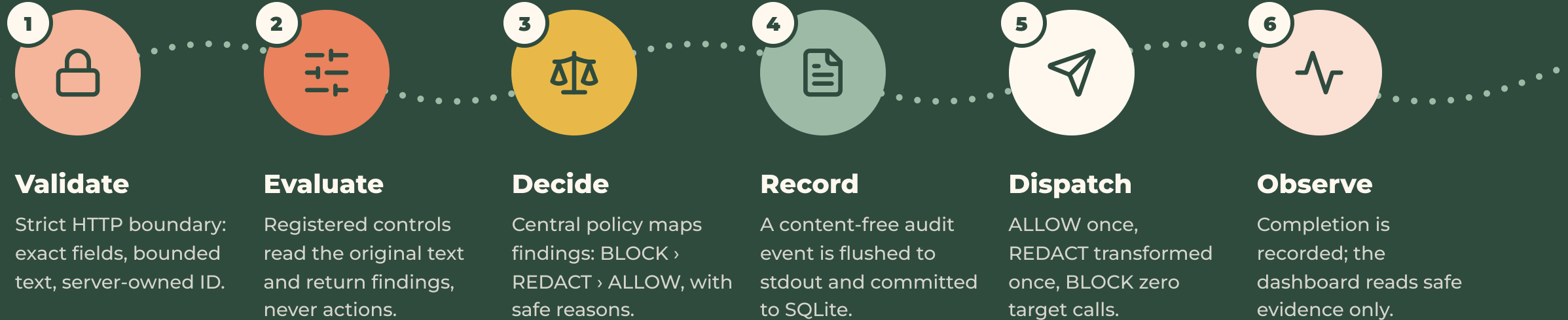
The control point must sit between intent and execution.



**The opportunity: make security a reusable service between every caller and every target.**

# How it works

*one request, step by step*



**If anything fails:**

invalid input

evaluation error

audit error



zero target calls

*fail closed!*

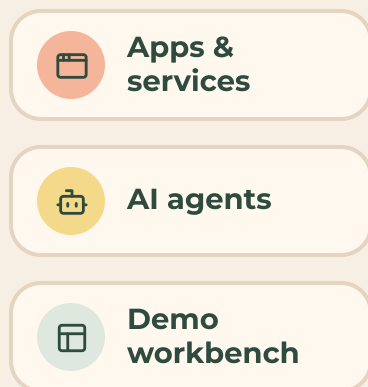
Every decision is structured, explainable and audited before eligible dispatch. Error responses carry fixed codes, never submitted text or exception details.

# Architecture

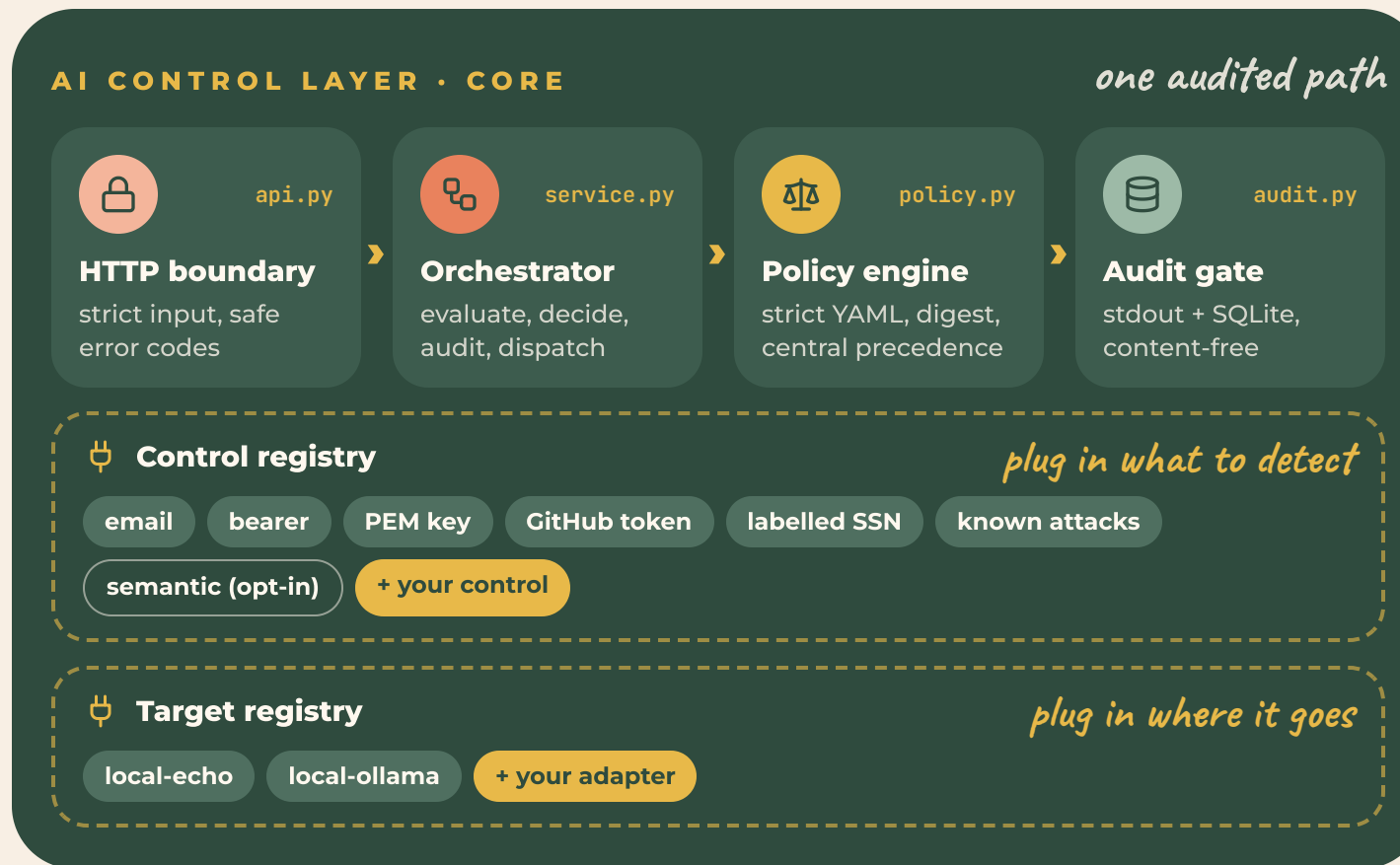
*a small core with plug-in edges*

Concrete detectors and targets are chosen only in **composition.py**. Policy, orchestration and audit have no provider-specific defaults.

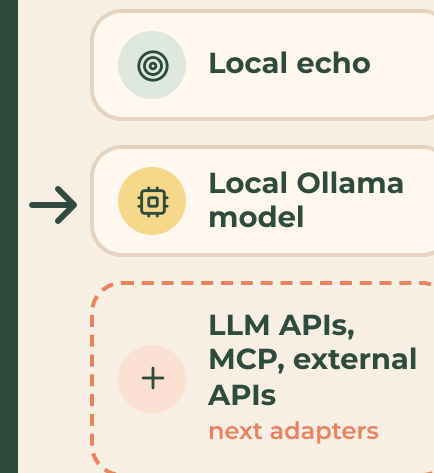
## callers



POST /v1/interactions



## targets



policy.yaml chosen per deployment



/dashboard read-only evidence

# Built to be reused

*swap the edges, keep the core*



**Proven in this repo:** five more controls and the local model target plugged in without a core redesign.

## Controls

WHAT TO DETECT

own finding codes

return findings only

registered at startup

## Policy

WHAT TO DO

one YAML per deployment

ALLOW · REDACT · BLOCK

validated, no fallback

## Targets

WHERE IT GOES

TargetAdapter contract

exact registered ID

called only after audit

## Core

STAYS THE SAME

validation + decisions

audit gate

reporting + dashboard

*for example:*



**Support chatbot**  
→ LLM

redact customer PII,  
block injection



**Coding agent** →  
**tools**

block leaked keys and  
tokens



**Internal  
knowledge  
assistant**

keep PII out of model  
prompts



**Agent** → **external  
API**

audit and gate every  
call

Example deployments reuse the same core; remote LLM, MCP and external API adapters are the next integrations, not part of today's build.

# Live today

*a focused pack, working end to end*



**7**

**registered  
controls**

6 deterministic (email, bearer, PEM, GitHub, labelled SSN, known attacks) + 1 opt-in semantic evaluator

**3**

**central  
actions**

ALLOW preserves, REDACT transforms, BLOCK stops

**2**

**target  
boundaries**

local echo and operator-configured loopback Ollama

**19**

**demo  
scenarios**

benign, deterministic, semantic and failure modes



**Deliberately bounded: local targets, strict startup policy and deterministic tests.**

## THE MOMENT OF CONTROL

# A simple outcome

*users can trust*

The decision engine makes the security posture visible in one line, while retaining the structured reasons behind it.



## ALLOW

*when*

No finding maps to a stronger action.

*then*

Forward the original interaction exactly once.



## REDACT

*when*

A supported PII finding needs protection.

*then*

Replace selected spans with [REDACTED] before forwarding.



## BLOCK

*when*

A credential or known attack indicator is detected.

*then*

Make zero target calls and return a safe reason.

Central precedence:

 BLOCK



 REDACT



 ALLOW

*the stronger action wins!*

# Coverage

## *with honest boundaries*

The pack recognizes precise supported shapes and keeps its claims narrow: testable, explainable and safe to extend.

| CONTROL                                  | DEFAULT                                                                                   | BOUNDARY                     |
|------------------------------------------|-------------------------------------------------------------------------------------------|------------------------------|
| <b>Email address</b><br>pii.email        |  REDACT  | bounded ASCII dot-atom       |
| <b>Labelled US SSN</b><br>pii.us_ssn     |  REDACT  | labelled, fictional syntax   |
| <b>Bearer / PEM / GitHub</b><br>secret.* |  BLOCK   | supported token / key shapes |
| <b>Known attack literals</b><br>attack.* |  BLOCK | 5 exact inert signatures     |
| <b>Semantic security</b><br>semantic.*   |  BLOCK | optional local evaluator     |

## Current limits

- ✦ **Output inspection** is not part of today's build.
- ✦ **Auth and budgets** are not enforced yet.
- ✦ **Policy reload** is not available; policy is fixed until restart.
- ✦ **Remote providers** are not integrated yet.
- ✦ **General secret and jailbreak detection** is not claimed.

*expansion points,  
not hidden claims*

# Evidence

*that helps operators move fast*

Every relevant decision becomes safe telemetry; the dashboard turns it into a living view of posture, timing and outcomes.

## Audit event

- ✓ server interaction ID
- ✓ UTC timestamp
- ✓ target + policy digest
- ✓ evaluated controls + enablement
- ✓ action + finding codes
- ✓ evaluation / invocation / total timing

## Security dashboard

*read-only overview*

**ALLOW**

posture



**REDACT**

protected



**BLOCK**

stopped



**UNKNOWN**

evidence gap



*privacy by design: audit keeps safe metadata only, never*



raw prompt content



matched values



spans



hashes



scores



exception text

# From live walkthrough to repeatable evidence

19

## code-owned scenarios

with expected explanations and  
actual-result reporting

*a mismatch stays  
visible, never  
rewritten as  
success*

## ALLOW

benign: safe request

## REDACT / BLOCK

deterministic: PII, token, key, attack

## BLOCK

semantic: injection, override, exfiltration

## EXPLAIN

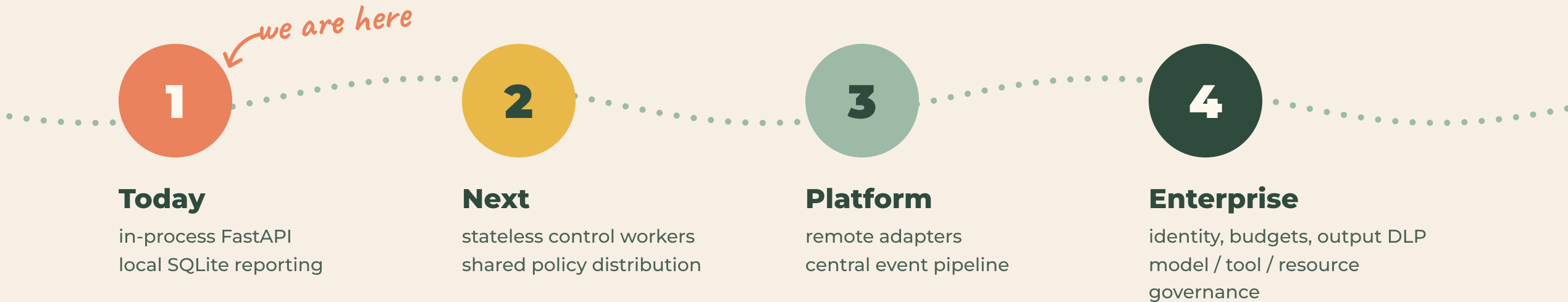
failure: evaluator / target unavailable

The local dashboard pairs a read-only security overview with an interaction workbench: operators see what was expected, what happened, why policy chose the action and whether the target completed.

# Scale the boundary first

*then scale the adapters*

The decision core stays small and composable: a credible path from local proof to platform service.



*scales cleanly:*

stateless controls

immutable registrations

policy-owned decisions

replaceable target adapters

Intentionally local today: synchronous request handling, loopback Ollama, single-node SQLite and no distributed queue. These are clear deployment boundaries, not architecture dead ends.

## WHY THIS WINS

# A control layer people can adopt before they trust it at scale.

It starts with a narrow, testable promise: no eligible AI interaction reaches a target without a policy decision and safe evidence.

**1,614**

unit + integration checks

**9**

Chromium journeys

**PASS**

fresh correctness +  
security reviews

**0**

target calls after BLOCK



*Protect the interaction. Preserve the evidence.  
Earn the next integration.*



AI Control Layer

[github.com/JakubPoltorak147/GoldmanSachsHackYeah2026](https://github.com/JakubPoltorak147/GoldmanSachsHackYeah2026)

Local proof today · scalable policy boundary tomorrow · HackYeah 2026